

Adaptive Edge–Cloud Anomaly Detection Framework

Track: Cloud Engineering **Challenge context:** National Research & Innovation Challenge Season 3, organized by 3SVK Official **Participant GitHub:** [Raghu2424](#) **Status:** Research documentation package based on participant-supplied project descriptions and reported results. This package does not claim code inspection, successful execution, or independent reproduction.

Purpose

The project compares three described anomaly-detection processing arrangements: centralized cloud inference, static edge inference, and resource-aware adaptive edge inference. The intended analysis considers detection metrics together with latency, CPU/memory utilization, and transmitted data. The paper is a provisional documentation draft; author identity, implementation details, and experimental records still require confirmation.

Research objectives

- Compare detection quality across the three described architectures.
- Compare latency, CPU and memory utilization, and transmitted bytes.
- Describe the proposed utility-based model/processing adaptation.
- Test whether adaptation provides a measurable advantage over static edge processing without assuming that it does.
- Document the artifacts and measurement protocol required for independent reproduction.

Architectures compared

1. **Centralized Cloud (Baseline A):** all input data reportedly sent to the cloud and processed with the HEAVY LSTM Autoencoder.
2. **Static Edge (Baseline B):** fixed STANDARD Autoencoder runs locally; anomaly events, rather than all input data, are reportedly sent onward.
3. **Adaptive Edge (proposed):** a described resource-aware utility mechanism chooses processing/model actions across LIGHT, STANDARD, and HEAVY classes; the reported configuration primarily selected STANDARD and sent anomaly events only.

These are summaries supplied by the participant; component paths and behaviors have not been checked against the repository's source code.

Model categories (as described)

Category	Model description	Project label
LIGHT	Isolation Forest anomaly detector	IsolationForestModel
STANDARD	Autoencoder-based anomaly detector	AutoencoderModel
HEAVY	Sequence-oriented LSTM autoencoder	LSTMAutoencoderModel

Exact model architecture, training procedure, inference costs, and configuration remain to be verified.

Reported findings — not independently reproduced

The project summary reports that static edge and adaptive edge have identical detection metrics: accuracy 0.8683, precision 0.1562, recall 0.2128, F1-score 0.1802, ROC-AUC 0.6128, and PR-AUC 0.1123. The centralized baseline's reported recall is 1.0000 and precision is 0.0797. The edge detection results therefore have a substantial recall limitation and should not be described as a universal improvement.

Reported transmitted data are 331,680 bytes for centralized cloud and 30,720 bytes for either edge approach; the arithmetic reduction is approximately 90.74%. Reported latency is 101.13 ms, 0.39 ms, and 0.50 ms for centralized, static edge, and adaptive edge, respectively. **These latency figures are not directly comparable:** centralized latency reportedly includes uplink, downlink, and inference, while edge latency represents local inference. The reported results and computed values are detailed in `research-paper.md` and `results/reported_results_transcribed.json`.

Limitations and reproducibility status

- Results are reported as coming from `results/final_results.json`, validation dated 2026-09-09; that JSON file was not available for inspection during document preparation.
- Experiments have not been independently rerun.
- The summary reports synthetic data, F1-based threshold calibration on validation data, fixed seeds, and Python 3.14.6 compatibility; none of these details has been checked against the actual artifacts.
- Dataset/generator, class prevalence, sample counts, confusion matrices, model hyperparameters, hardware, network conditions, run count, and uncertainty measures are not supplied.
- The adaptive and static detection values are identical; adaptation has not demonstrated a detection improvement in the reported run.

- The official repository structure and contribution instructions were not inspected because web access was unavailable under the current source settings. Check the official repository manually before placing this packet.

Documentation folder structure

```
3svk-submission/
  README.md
  research-paper.md
  diagrams/
    architecture.md
  references/
    literature_review.md
    research_gap.md
  results/
    reported_results_transcribed.json
```

The folder name is a packaging label for this draft. Before copying files into the official repository, follow its actual contribution path and naming requirements. If its root already has a README, preserve the repository README and place this packet in the contributor location specified by the maintainers.

Artifacts to verify before publication

1. Actual repository source and configuration, including `src/adaptive/`, `src/edge/`, `src/cloud/`, `src/evaluation/`, and `default_config.yaml`.
2. Original `results/final_results.json`, logs, and evaluation scripts.
3. Dataset or synthetic-data generator, labels, class balance, partitions, and split protocol.
4. Model definitions, checkpoints or deterministic training instructions, preprocessing, hyperparameters, and threshold-selection code.
5. Adaptive utility equation, weights, resource inputs, decisions, model-selection traces, and event/transmission behavior.
6. Metric implementation, confusion matrices, ROC/PR computations, and thresholds.
7. Hardware/software environment, actual Python version, dependencies, network conditions, timing boundaries, byte-accounting rules, and resource-monitoring procedure.
8. Repeated-run results and real-world validation, where feasible.
9. Official challenge submission rules, folder path, filename requirements, deadline, and event-platform deliverables.

Citation instructions

The author and publication details for this project are not finalized. Do not cite this draft as a published paper or invent a venue, DOI, date, or author list. Once authorship and publication status are confirmed, use the actual metadata in the target style. A neutral template is: [Author surname(s)], [Initials]. ([Year]). Adaptive Edge-Cloud Anomaly Detection Framework: A Resource-Aware Comparison of Centralized and Edge-Based Processing. [Actual venue or repository status]. [DOI or stable repository URL, if assigned]. The literature sources and their verified/needs-verification status are catalogued in references/literature_review.md.

Contributing or reproducing

This packet does not assume that a particular run command, test command, package manager, or environment file exists. To reproduce the study, first inspect the project README and configuration; obtain the dataset/generator and dependencies; confirm documented commands from the repository; then run the evaluation only after validating the data split and configuration. Record the checked-out revision, environment, seeds, system/network setup, and complete output artifacts. Do not add command examples until they have been confirmed from the real project.

For documentation changes, keep participant-reported results clearly labelled until reproduced, preserve the latency-boundary caveat, and avoid claims of significance or adaptive advantage without supporting experiments.

3SVK submission context

The participant-provided workflow is to fork the official repository, add the required research documentation, commit the changes, open a Pull Request (PR), then submit the GitHub profile link and PR screenshot on the event platform. Official project links:

- Official research repository: <https://github.com/3SVK-Official/3SVK-Research-Series-Season-3>
- Official challenge website: <https://sites.google.com/view/3svk-research-s3/home>
- Participant profile: <https://github.com/Raghu2424>

Verify the challenge deadline and exact deliverables on the official event page. Opening a PR does not mean it has been merged or accepted.